



## **Chapter S3F: PyMongo**

### **CONTENT**

| <b>No.</b> | <b>Topic</b>                            | <b>Syllabus Guide</b> |
|------------|-----------------------------------------|-----------------------|
| <b>1</b>   | <b>Accessing MongoDB using PyMongo</b>  | <b>3.3.8</b>          |
| <b>2</b>   | <b>Inserting Documents</b>              |                       |
| <b>3</b>   | <b>Importing Data from Files</b>        |                       |
| <b>4</b>   | <b>Viewing and Querying Documents</b>   |                       |
| <b>5</b>   | <b>Query Operators</b>                  |                       |
| <b>6</b>   | <b>Result Manipulation</b>              |                       |
| <b>7</b>   | <b>Updating Documents</b>               |                       |
| <b>8</b>   | <b>Deleting Documents</b>               |                       |
| <b>9</b>   | <b>Dropping Collections / Databases</b> |                       |
| <b>10</b>  | <b>Programming Exercises</b>            |                       |
| <b>11</b>  | <b>References</b>                       |                       |



## 1. Accessing MongoDB using PyMongo

MongoDB is a NoSQL (document-based) database. In Python, MongoDB is accessed using **PyMongo**. Python programs connect to MongoDB using MongoClient and interact with databases and collections as Python objects.

Assume the MongoDB server is already running. The program connects to the MongoDB server and outputs the databases currently in the MongoDB server.

### Program 1: access.py

```
1 import pymongo
2 client = pymongo.MongoClient("127.0.0.1", 27017)
3 print("The databases in the MongoDB server are:")
4 print(client.list_database_names())
5 client.close()
```

- MongoClient(host, port) connects to the MongoDB server
- list\_database\_names() returns a Python list
- Always close the connection

## 2. Inserting Documents

MongoDB stores data as documents (Python dictionaries) inside collections. MongoDB waits until at least one document is inserted before it actually creates the database and collection.

### Program 2: insert.py

```
1 from pymongo import MongoClient
2 client = MongoClient("localhost", 27017)
3 db = client["entertainment"]
4 coll = db["movies"]
5 coll.insert_one({"_id":101, "title":"Johnny Maths",
6 "genre":"comedy"})
7 coll.insert_many([
8     {"title":"Badman", "genre":["adventure"], "year":2015},
9     {"title":"Averages", "genre":["science fiction",
10 "adventure"], "year":2017},
11     {"title":"Future Maths", "genre":["science fiction"],
12 "year":2018}])
13 coll_names = db.list_collection_names()
14 print ("Collections in entertainment database: ",coll_names)
15 client.close()
```



- Inserting one document using the `insert_one()` method, and a list of documents using the `insert_many()` method
- MongoDB automatically creates a unique `_id` (ObjectID) if not provided. If a document is inserted with a user-defined `_id`, it must be unique. Inserting another document with the same `_id` will result in a “duplicate key error”.
- A field may store a list.
- `list_collection_names()` method retrieves a list of all collection names within a database

### Quiz 1

Write a Python program to ask for one movie title and the year of movie, then insert the document into the movie collection. Assume no genre is given. Save as filename: `quiz1.py`. Extend the program to include genres (where movies can have none or multiple genres). Save as filename: `quiz1_extend.py`

## **3. Importing Data from Files**

### **3.1 Import from text/CSV file**

**Input File:** `input.txt`

Add on your own data to this input file which contains names and ages

```
Amanda,45
Bala,28
Charlie,33
Devi,29
Elon, 51
Frank, 38
...
```

**Program 3:** `importFile.py`

```
1 import pymongo, csv
2 client = pymongo.MongoClient("127.0.0.1", 27017)
3 coll = client["entertainment"]["users"]
4 file = open('input.txt','r')
5 csv_reader = csv.reader(file, delimiter=',')
6 for row in csv_reader:
7     coll.insert_one({"name":row[0].strip(), "age":int(row[1].strip())})
8 file.close()
9 client.close()
```



### 3.2 Import from JSON file

**JSON file:** data.json

```
[
  {
    "name": "Amanda",
    "age": "45"
  },
  {
    "name": "Bala",
    "age": "28"
  },
  {
    "name": "Charlie",
    "age": "33"
  },
  {
    "name": "Devi",
    "age": "29"
  }
]
```

**Program 4:** loadjson.py

```
1 import pymongo, json
2 client = pymongo.MongoClient("127.0.0.1", 27017)
3 with open('data.json') as file:
4     data = json.load(file)
5 client['entertainment']['users'].insert_many(data)
6 client.close()
```

### 4. Viewing and Querying Documents

**Program 5:** view.py

```
1 import pymongo
2 client = pymongo.MongoClient("127.0.0.1", 27017)
3 coll = client["entertainment"]["movies"]
4 result = coll.find()
5 print("Documents in movie collection:")
6 for doc in result:
```



```
7     print(doc)
8     print("Number of items in movie collection:", coll.count_documents({}))
9
10    result = coll.find({'genre': ['adventure']})
11    print("Movies with adventure genre:")
12    for doc in result:
13        print(doc)
14
15    query2 = {'genre': 'adventure', 'year': {'$gt': 2016}}
16    result = coll.find(query2)
17    print("Titles of movies with adventure genre after 2016:")
18    for doc in result:
19        print(" - " + doc.get('title'))
20    print("There are", coll.count_documents(query2), "movies in the list above.")
21    client.close()
```

- The `find()` method returns a cursor of all the documents in collection
- Each document behaves like a Python dictionary
- Use `get()` to avoid key errors
- Use `count_documents()` to get the number of documents in the collection
- `query2` can be rewritten using the `$and` operator: `query2 = {'$and': [{'genre': 'adventure'}, {'year': {'$gt': 2016}}]}`

## 5. Query Operators

The following is a list of commonly used query operators.

| Comparison operators           |                                                            |
|--------------------------------|------------------------------------------------------------|
| \$eq                           | Equal to                                                   |
| \$gt                           | Greater than                                               |
| \$gte                          | Greater than or equal to                                   |
| \$lt                           | Less than                                                  |
| \$lte                          | Less than or equal to                                      |
| \$ne                           | Not equal to                                               |
| Logical / membership operators |                                                            |
| \$in                           | Value is in a specified list                               |
| \$nin                          | Value is not in a specified list                           |
| \$or                           | Logical OR                                                 |
| \$and                          | Logical AND                                                |
| \$not                          | Logical NOT                                                |
| \$exists                       | Checks whether a field exists (not the value of the field) |



**Program 6: view2.py**

```
1 import pymongo
2 client = pymongo.MongoClient("127.0.0.1", 27017)
3 coll = client["entertainment"]["movies"]
4
5 result = coll.find()
6 print("Documents in movie collection:")
7 for doc in result:
8     print(doc)
9 print("Number of items in movie collection:", coll.count_documents({}))
10
11 result = coll.find({'genre':{'$in':['adventure', 'comedy']}})
12 print("Movies with adventure or comedy genre:")
13 for doc in result:
14     print(doc)
15
16 query2 = {'genre': {'$exists':False}}
17 result = coll.find(query2)
18 print("Movies without genre:")
19 for doc in result:
20     print(" - " + doc.get('title'))
21
22 result = coll.find_one({'year':{'$eq':2017}})
23 print("One movie that was released in 2017")
24 print(result)
25 client.close()
```

- `find_one()` returns one document that matches the condition

Quiz 2: Modify lines 11 and 12 to find all movies without adventure and comedy genres.

.....  
.....

Quiz 3: Modify lines 16 to 20 such that for all movies with year, print out the movie title and how many years ago the movie was released.

.....  
.....  
.....  
.....  
.....



Quiz 4: Modify lines 22 to 24 to print out all movies released before 2017.

.....  
.....  
.....

## 6. Result Manipulation

| Method               | Description                                                                         | Example of usage                                                                                                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>skip()</code>  | skip specified number of documents from the result set                              | <code>result= coll.find().skip(5)</code><br>Skips the first 5 documents.                                                                                                                                     |
| <code>limit()</code> | limit the number of documents returned in the result set                            | <code>result=coll.find().limit(10)</code><br>Limits result set to the first 10 documents,                                                                                                                    |
| <code>sort()</code>  | sort the documents based on one of more fields (1 for ascending, -1 for descending) | <code>result = coll.find().sort('title', 1)</code><br>Sorts documents by title in ascending order.<br><br><code>result = coll.find().sort('name', -1)</code><br>Sorts documents by name in descending order. |

Combining all 3 methods:

```
result = coll.find({"genre": "adventure"}).sort('title', 1).skip(10).limit(5)
```

This finds documents with the genre "adventure," sorts them by title in ascending order, skips the first 10 documents, and limits the result set to the next 5 documents.

## 7. Updating Documents

| Update operators        |                                                                                                                                                      |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>\$set</code>      | Modify / add field                                                                                                                                   |
| <code>\$unset</code>    | Remove field                                                                                                                                         |
| <code>\$push</code>     | Add a value to an array field.<br>If the field does not exist, MongoDB creates it as an array.                                                       |
| <code>\$addToSet</code> | Add a value only if value is not present                                                                                                             |
| <code>\$pull</code>     | Removes all occurrences of a value from an array field                                                                                               |
| <code>\$inc</code>      | Increments (or decrements) a numeric field by a specified amount.<br>If the field does not exist, it will be created and set to the increment value. |

Update operators modify only the specified fields and do not replace the entire document.



**Program 7: update.py**

```
1 import pymongo
2 client = pymongo.MongoClient("127.0.0.1", 27017)
3 coll = client["entertainment"]["movies"]
4 result = coll.find() # Display all movies
5 print("Documents in movies collection:")
6 for doc in result:
7     print(doc)
8 # Set all the year values greater than 2016 to be 2015
9 search = {'year':{'$gt':2016}}
10 update = {'$set':{'year':2015}}
11 coll.update_one(search, update)
12 result = coll.find() # Display result after update one
13 print("Documents in movies collection after update one:")
14 for doc in result:
15     print(doc)
16 coll.update_many(search, update)
17 result = coll.find() # Display result after update many
18 print("Documents in movies collection after updating all:")
19 for doc in result:
20     print(doc)
21 # Remove field 'year'
22 search = {'year':{'$eq':2018}}
23 update = {'$unset':{'year':0}}
24 coll.update_many(search, update)
25 # Adds "comedy" to the genre list of Badman
26 search = {"title": "Badman"}
27 update = {"$push": {"genre": "comedy"}}
28 coll.update_one(search, update)
29 # Removes "adventure" from the genre list
30 search = {"title": "Averages"}
31 update = {"$pull": {"genre": "adventure"}}
32 coll.update_one(search, update)
33 # Increases the value of year by 1
34 search = {"title": "Johnny Maths"}
35 update = {"$inc": {"year": 1}}
36 coll.update_one(search, update)
37 client.close()
```

- The `update_one()` method modifies the first document that matches the query
- The `update_many()` method modifies all documents that matches the query
- Insert several documents that match the search condition to clearly see the effect of `update_many()`





Quiz 5: Modify code to add comedy genre to all movies that currently have no genres.

.....

Quiz 6: Modify code to remove the genre field to all movies that currently have adventure as its genre or one of its genres.

.....

To ensure no two identical documents are inserted, you can check before inserting:

```
new_doc = {"title": "New Movie", "genre": ["action"], "year": 2023}
if coll.count_documents(new_doc) == 0:
    coll.insert_one(new_doc)
else:
    print("Document already exists!")
```

## 8. Deleting Documents

### Program 8: delete.py

```
1  import pymongo
2  client = pymongo.MongoClient("127.0.0.1", 27017)
3  coll = client["entertainment"]["movies"]
4
5  result = coll.find()
6  print("Documents in movies collection:")
7  for doc in result:
8      print(doc)
9
10 coll.delete_one({'year':2015})
11 result = coll.find()
12 print("Documents in movies collection after deleting one:")
13 for doc in result:
14     print(doc)
15
16 coll.delete_many({'year':2015})
17 result = coll.find()
18 print("Documents in movies collection after deleting all:")
19 for doc in result:
20     print(doc)
21
22 client.close()
```

- The `delete_one()` method deletes the first document that matches the condition
- The `delete_many()` method deletes all the documents that match the condition



Quiz 7: Modify code to delete all movies with adventure as its genre or one of its genres.

.....

## 9. Dropping Collections / Databases

To clear the collection, you can write a program similar to program 8 below.

**Program 9:** remove.py

```
1  import pymongo
2  client = pymongo.MongoClient("127.0.0.1", 27017)
3  db = client["entertainment"]
4  coll = db["tv"]
5  coll.insert_one({"title":"X Man", "genre":"science fiction"})
6  coll.insert_one({"title":"Fresh from the boat","genre":"comedy"})
7  coll.insert_one({"title":"", "genre":"comedy"})
8  coll.insert_one({"genre":"comedy"})
9  result = coll.find()
10 print("Documents in tv collection:")
11 for doc in result:
12     print(doc)
13 print("Number of items in tv collection:", coll.count_documents({}))
14 db.drop_collection("tv")
15 result = coll.find()
16 print("After tv collection is dropped:")
17 for doc in result:
18     print(doc)
19 print("Number of items in tv collection:", coll.count_documents({}))
20 client.close()
```

- Use `client.drop_database("entertainment")` to remove the entire entertainment database. This will remove all the collections and documents within it.



## 10. Programming Exercises

### Task 1: Inventory GEM (Adapted from ACJC Prelim 2020 Paper 2 Question 3)

A text file `INVENTORY_SERIAL.txt` contains the inventory data for a certain electronics store. Each line in the updated file contains *tab-delimited* data that shows the serial number, product name, product type, purchase price, selling price and quantity available.

Each line is in the format:

`Serial_No\tName\tType\tPurchase_Price\tSelling_Price\tQuantity`  
where `\t` represents the tab character.

Sample file of `INVENTORY_SERIAL.txt`

```
1PH1 Bapple 8 Phone 745.55 1200.00 20
1PH2 Bapple 9 Phone 825.35 1348.00 20
1PH3 Bapple 10 Phone 848.22 1399.95 15
1PH4 Bapple 11 Phone 836.12 1488.95 18
1PH5 Bapple 12 Phone 736.00 1588.85 15
1RT1 Elink1 200 Router 48.00 199.95 15
1RT2 Elink1 300 Router 58.00 218.95 16
2RTA Dnetwork 500! Router 28.00 99.95 19
2RT2 Dnetwork 750 Router 38.00 129.95 21
1LT1 ThinkerpadX 13 Laptop 648.65 1200.95 5
1LT2 ThinkerpadX 15@ Laptop 799.15 1399.95 8
2LT1 Surface Elite 5 Laptop 499.00 999.95 8
2LT2A Surface Elite 5 Laptop 699.00 1399.95 9
2LT3 Surface Elite 7 Laptop 698.90 1511.95 sold out
1TB Universe Tab A Tablet 399.90 699.95 12
1TB2 Universe Tab B Tablet 449.00 799.95 -14
1TB3 Universe Tab C Tablet 432.00 799.95 15
```

**1.1** Write program code to insert the data from `INVENTORY_SERIAL.txt` into a NoSQL database `OUTLETS` under the collection `GEM`.

Save as filename `gem_task1_1_<your name>.py`.

**1.2** The database administrator wants to validate that the store manager did not make any errors when he edited the text file. Write program code to check that the database conforms to the below specifications:

- `Serial_No` consists of one digit followed by two letters, followed by one digit (e.g. 1AB7);
- `Name` consists of only letters, digits and spaces;
- `Quantity` is a positive integer.

Any document that has an error should be removed from the database.

You may assume data fields not specified above are error free.

Display the documents that were removed and save them to a text file in the format:

`Serial_No\tName\tType\tPurchase_Price\tSelling_Price\tQuantity`

Save as filename `gem_task1_2_<your name>.py`.



**Task 2: Mobile Phones** (Adapted from JPJC Prelim 2021 Paper 2 Question 4)

JP Mobile sells mobile phones and manages its inventory using a NoSQL database. Information about the mobile phones is stored in the JSON file `items.JSON`.

The following fields are recorded:

- brand of mobile phone,
- model,
- colour(s) available,
- price in dollar,
- quantity in stock.

Part of the file is as shown:

```
[
  {
    "brand": "orange",
    "model": "22",
    "colour": "black",
    "price": 888,
    "quantity": 12
  },
  {
    "brand": "solo",
    "model": "A11",
    "colour": "black",
    "price": 699,
    "quantity": 8
  },
  {
    "brand": "yala",
    "model": "M3",
    "colour": "white",
    "price": 399,
    "quantity": 10
  },
]
```

**2.1** Write program code to import the information from the JSON file into a MongoDB database. Save the information under the `phone` collection in the `jp_mobile` database.

Ensure that the collection **only** stores the information from the JSON file.

Save as filename `mobile_task2_<your name>.py`.

**2.2** The shop decides to include one or more free gifts for new batches of mobile phones it sells. Add to your program code for a user to insert information of a mobile phone by getting user input of the following: brand, model, colour, price, quantity, free gift(s).

Your code should allow user to input one or more free gifts.

If a phone's brand, model and colour already exists, add the new quantity to the existing quantity in the database, and replace the existing price with the new price.

Run your program and insert the following 2 documents:

| No. | Brand  | Model | Colour | Price | Quantity | Free gift(s)        |
|-----|--------|-------|--------|-------|----------|---------------------|
| 1   | orange | 22    | black  | 900   | 11       | power bank          |
| 2   | solo   | A33   | red    | 1300  | 7        | power bank, earbuds |

Add your program code to `mobile_task2_<your name>.py`.



**2.3** Write a function `display_all` that will display all the information in the phone collection under these fields: brand, model, colour, price, quantity, free gift(s).

If no free gift comes with the phone, print a `None` statement.

Include a final statement that shows the total number of documents in the collection.

Run the `display_all` function to show your output.

Add your program code to `mobile_task2_<your name>.py`.

### **Task 3: ElectroMart**

ElectroMart is a retail store that sells electronic products. The store uses a MongoDB NoSQL database to manage its products, customers, and orders. You are required to write Python programs using PyMongo to implement the system described below.

The MongoDB database is named **electromart** and contains the following collections:

Each product document stores:

- `product_id` (string, unique)
- `name` (string)
- `category` (string)
- `price` (decimal number)
- `stock` (integer)

Each customer document stores:

- `customer_id` (string, unique)
- `name` (string)
- `email` (string)
- `address` (string)

Each order document stores:

- `order_id` (string, unique)
- `customer_id` (string)
- `timestamp` (date/time)
- `product_id` (string)
- `quantity` (integer)

Each order records only one product.

**3.1 Database Creation:** Write Python program code to:

- connect to MongoDB
- create the `electromart` database
- create the `products`, `customers`, and `orders` collections

Save your program file as: `electromart_task3_<your name>.py`



**3.2 Importing Data:** Two JSON files are provided:

- `products.json`
- `customers.json`

Write program code to:

1. import all product documents into the `products` collection
  - ensure that all product prices are **positive**
2. import all customer documents into the `customers` collection
  - ensure that **no two customers have the same email address**

Any customer documents with duplicate emails should **not be inserted**.  
Display the list of customers that were rejected.

**3.3 Placing an Order:** Write a function `place_order (customer_id, product_id, quantity)`. The function should:

1. check if the requested quantity is available in stock
2. if sufficient stock exists:
  - insert a new document into the `orders` collection
  - deduct the ordered quantity from the product stock
3. if stock is insufficient:
  - display the message `InsufficientStockError`

**3.4 Sales Report:** Write a function `generate_sales_report()` that calculates:

- total revenue for each product category
- total number of orders made by each customer

Display the report in a readable format.

**3.5 Retrieve Products by Category:** Write a function `get_products_by_category(category)` that retrieves and displays all products belonging to the given category.

**3.6 Find Customer by Email:** Write a function `find_customer_by_email(email)` that searches for a customer using their email address and displays the customer's details if found.

**3.7 Retrieve Orders by Customer:** Write a function called `get_orders_by_customer (customer_id)` that retrieves and displays all orders placed by the given customer.



**3.8 Menu System:** Implement a menu-driven program that repeatedly displays the following options:

**ElectroMart Inventory Management System**

1. Import Data
2. Place Order
3. Generate Sales Report
4. Get Products by Category
5. Find Customer by Email
6. Get Orders by Customer
7. Exit

Your program should:

- accept user input
- execute the corresponding function
- handle invalid input appropriately
- allow the user to exit gracefully

Include comments to explain your program code.

Save your file as: `electromart_task3_<your name>.py`.

**Task 3 ElectroMart Extension**

**3.9 Multi-Product Orders:** In the current system, each document in the orders collection stores only one product per order. The store now wants to allow multiple products per order. The new structure of each order document should be:

```
{
  "order_id": "...",
  "customer_id": "...",
  "timestamp": "...",
  "items": [
    {"product_id": "...", "quantity": ...},
    {"product_id": "...", "quantity": ...}
  ]
}
```



Write program code to:

1. Retrieve **all existing documents** in the orders collection.
2. Modify each document so that:
  - the existing product\_id and quantity fields are converted into a **single-item list** under items
  - the timestamp field is **preserved and not modified**
3. Update the modified documents back into the orders collection.

**3.10 Modified Order Function:** Modify the existing `place_order` function so that it supports multi-product orders. The new function should have the following header: `place_order(customer_id, items)` where `items` is a list of dictionaries, for example: `[{"product_id": "P101", "quantity": 2}, {"product_id": "P103", "quantity": 1}]`

The function should:

1. Check that sufficient stock exists for all products in the order.
2. If any product has insufficient stock:
  - reject the entire order
  - display an error message indicating the product involved
3. If all products have sufficient stock:
  - insert a new order document into the orders collection using the new structure
  - deduct the ordered quantities from the relevant products
  - display a confirmation message

**3.11 Display Orders by Customer:** Modify the `get_orders_by_customer(customer_id)` function so that it:

1. Retrieves all orders placed by the given customer.
2. Displays, for each order:
  - order ID
  - timestamp
  - all product IDs and quantities contained in the order

The output should clearly show all items within each order.

## 11. References

| Content             | Link                                                                                                                                                                              |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MongoDB/<br>PyMongo | <a href="https://www.mongodb.com/company/what-is-mongodb">https://www.mongodb.com/company/what-is-mongodb</a>                                                                     |
|                     | <a href="https://www.mongodb.com/docs/languages/python/pymongo-driver/current/">https://www.mongodb.com/docs/languages/python/pymongo-driver/current/</a>                         |
|                     | <a href="https://www.mongodb.com/docs/languages/python/pymongo-driver/current/get-started/">https://www.mongodb.com/docs/languages/python/pymongo-driver/current/get-started/</a> |